UNIVERSIDAD PANAMERICANA

Campus México

Materia: Sistemas Operativos

Profesor: Iram Maximiliano Lopez Guerrero

Fecha de entrega: 22 de abril de 2026

Ciclo: Enero 2026

"Proyecto de Sistemas Operativos"

Miembros del Equipo		
ID	**Nombre (ap.Paterno, ap.Materno, nombre)**	**Carrera**
0246717	Flores Alcocer Andoreni Claudio	LIDCI

Mini escáner de red inspirado en nmap implementado en C++ sobre macOS

1. Resumen

El proyecto consiste en el desarrollo de una herramienta de escaneo de red inspirada en nmap, implementada en C++ sobre un sistema tipo macOS. La aplicación permite realizar un escaneo básico de puertos TCP sobre una dirección IP o nombre de host, clasificando su estado (abierto o cerrado) e identificando de forma sencilla el servicio asociado a cada puerto abierto mediante una tabla estática de servicios bien conocidos.

El proyecto está dirigido a administradores de sistemas, estudiantes y profesionales de ciberseguridad que necesitan una herramienta ligera para reconocimiento de red o para fines educativos. Su razón de ser es profundizar en el manejo de sockets, el funcionamiento de los protocolos de red y la interacción con el sistema operativo desde C++, así como ejercitar conceptos de concurrencia (hilos, sincronización, escritura concurrente en memoria) propios de la materia de Sistemas Operativos. El resultado es una utilidad práctica que sirve como base para futuros desarrollos de seguridad y automatización de tareas de reconocimiento.

2. Tiempo estimado de desarrollo

El tiempo total estimado para el desarrollo completo del proyecto se sitúa entre 40 y 60 horas a lo largo del semestre, distribuidas de la siguiente manera:

- Análisis y diseño: 8–10 horas.
- Implementación del núcleo de red (sockets, escaneo básico): 15–20 horas.
- Funcionalidades avanzadas (concurrencia, manejo flexible de rangos, futura detección de servicios con banners): 10–15 horas.
- Pruebas, validación y documentación: 8–10 horas.

3. Tecnologías usadas

3.1 Lenguaje de programación

C++ estándar C++17 o superior, ejecutándose sobre un sistema macOS.

3.2 Librerías

- **Biblioteca estándar de C++ (STL):** uso de std::string, std::vector, std::map, std::thread, std::mutex, std::chrono y manejo básico de excepciones.
- **API de sockets Berkeley (POSIX):** llamadas como socket(), connect(), close(), getaddrinfo() y setsockopt() para la comunicación de red de bajo nivel.

3.3 Herramientas extra

- **Compilador:** g++ o clang++.
- **Sistema operativo base:** macOS.
- **Sistema de construcción:** Makefile o CMake.
- **Control de versiones:** git + repositorio remoto en GitHub.
- **Herramientas de prueba y monitoreo (solo para validación):** nmap, ping, tcpdump y/o Wireshark.

4. Justificación tecnológica

Para el desarrollo de este escáner de red se utiliza C++ debido a su buen equilibrio entre rendimiento, control de bajo nivel y soporte de abstracciones de alto nivel mediante la biblioteca estándar. La API de sockets POSIX se elige porque expone directamente la interfaz de red que provee el sistema operativo, lo que permite entender de manera más fiel cómo se crean, gestionan y cierran conexiones TCP desde el espacio de usuario. Esto resulta fundamental en un escáner de puertos, donde es necesario controlar tiempos de espera, interpretar códigos de error y manejar múltiples conexiones concurrentes.

El uso de la biblioteca estándar de C++ permite manejar colecciones de puertos, rangos de direcciones IP y cadenas de texto de forma segura y eficiente, sin agregar dependencias externas de alto nivel que oculten el funcionamiento interno. Las herramientas de apoyo como git y GitHub facilitan el control de versiones y la colaboración, mientras que nmap y las herramientas de análisis de tráfico se utilizan únicamente como referencia para validar el comportamiento del escáner desarrollado.

5. Problemática que resuelve y público objetivo

El proyecto resuelve la necesidad de contar con una herramienta ligera y sencilla para reconocimiento de red y detección de puertos abiertos en hosts específicos. Este tipo de herramienta es útil en tareas de administración de sistemas, auditorías básicas de seguridad y prácticas de laboratorio en cursos de redes y ciberseguridad.

El escáner está dirigido a:

- Estudiantes de informática, ingeniería de software, redes y ciberseguridad.
- Administradores de sistemas que requieran una herramienta pequeña para pruebas rápidas.
- Personas interesadas en entender cómo funcionan internamente los escáneres de puertos.

La razón de ser del proyecto es doble: por un lado, aprender y practicar el uso de C++ y sockets en un entorno macOS, ejercitando además conceptos centrales de sistemas operativos como

hilos y sincronización; por otro, obtener una herramienta funcional que pueda utilizarse en ejercicios de reconocimiento de red y que sirva como base para futuras extensiones (nuevos tipos de escaneo, análisis de servicios, automatización).

6. Razón de uso de las librerías

Se usa la API de sockets POSIX para resolver la problemática de establecer conexiones TCP a múltiples puertos, ya que esta interfaz permite configurar timeouts, manejar estados de conexión y diferenciar entre distintos tipos de error (rechazo, timeout, fallo de resolución, etc.). Esto es clave para clasificar correctamente el estado de los puertos escaneados.

La biblioteca estándar de C++ se utiliza para representar las listas de puertos, rangos de IPs y resultados del escaneo, así como para gestionar hilos de ejecución en la fase de concurrencia. Esta combinación ofrece suficiente control de bajo nivel para entender qué ocurre en el sistema operativo, pero con herramientas que facilitan la escritura de código mantenible y relativamente seguro.

7. Desarrollo del proyecto

7.1 Análisis y diseño

Para el segundo parcial se mantuvo la estructura mínima de un escáner TCP tipo connect scan establecida en el primer entregable y se concentraron las mejoras en tres áreas concretas: (a) flexibilidad en la especificación de puertos a escanear, (b) un formato de salida más informativo y legible, y (c) la introducción de concurrencia para reducir el tiempo total de escaneo en rangos amplios.

De las funcionalidades listadas originalmente en el plan del primer parcial, se descartó deliberadamente la lectura de banners y el descubrimiento automático de hosts activos. La justificación es doble. Primero, la planificación de entregas solicita para este parcial una de entre "concurrencia o detección de servicios con banners", no ambas; priorizar concurrencia alinea mejor con los objetivos de la materia de Sistemas Operativos, ya que fuerza a trabajar con hilos, sincronización y escritura concurrente en memoria. Segundo, el descubrimiento de hosts está marcado como opcional en el propio plan. El resultado es un entregable más profundo en el eje que realmente interesa a la materia.

La arquitectura se descompuso en cuatro clases con responsabilidades separadas:

- **ScannerApp** orquesta el flujo general del programa.
- **ArgumentParser** interpreta y valida la línea de comandos.
- **PortScanner** ejecuta el escaneo concurrente sobre sockets TCP.
- **ResultFormatter** imprime la tabla de resultados y el resumen final.

Los tipos compartidos (Args, PortResult, PortState) se aislaron en un encabezado Types.hpp para desacoplar las clases entre sí.

7.2 Implementación del núcleo de red

El núcleo de red reside en PortScanner.cpp. El flujo comienza con una única resolución del host mediante getaddrinfo(), de la cual se obtiene una estructura sockaddr_in base que contiene la dirección IPv4 resuelta. Esta estructura se copia (no se comparte por referencia) a cada hilo, de modo que cada hilo puede modificar independientemente el campo sin_port sin incurrir en una condición de carrera sobre la dirección de destino.

Por cada puerto a escanear, el hilo ejecuta la siguiente secuencia:

1. socket(AF_INET, SOCK_STREAM, 0) — crea un socket TCP IPv4.
2. connect(sock, ...) — intenta el three-way handshake de TCP contra la dirección y puerto objetivo.
3. Clasifica el resultado: si connect() devuelve 0, el puerto se marca como OPEN; cualquier otro valor se marca como CLOSED.
4. close(sock) — libera el descriptor de archivo asociado al socket.

Se optó por no configurar un timeout explícito con setsockopt() o select(). Esto implica que puertos filtrados por un firewall se reportan como CLOSED en lugar de distinguirse como un estado FILTERED independiente. Esta decisión mantiene acotado el alcance del parcial: la distinción entre CLOSED y FILTERED y los timeouts configurables forman parte del entregable final del proyecto.

7.3 Implementación del escáner de puertos con concurrencia

El patrón implementado es un hilo std::thread por puerto. Dado que el número total de puertos a escanear suele ser conocido y acotado (típicamente entre decenas y algunos miles), no se requiere una cola de trabajo compartida ni un thread pool. Esta decisión simplifica drásticamente el código y resulta suficiente para los rangos típicos del proyecto.

El vector de resultados se pre-dimensiona antes de lanzar cualquier hilo. Cada hilo escribe exclusivamente en el índice que le corresponde (results[i]), por lo que no existe contienda de datos sobre el contenedor: el estándar de C++ garantiza que escrituras a elementos distintos de un contenedor pre-dimensionado son independientes siempre y cuando no se invoque resize, push_back o similares durante la ejecución concurrente.

El único recurso verdaderamente compartido entre hilos es std::cerr, al que los hilos podrían escribir si socket() falla por algún motivo (por ejemplo, agotamiento de descriptores de archivo). Para evitar que las líneas de error se intercalen de forma ilegible, se protege std::cerr con un std::mutex global (g_cerrMutex) mediante std::lock_guard. No se usa mutex para std::cout,

porque la impresión final la realiza ResultFormatter una vez que todos los hilos han terminado (join), por lo que ya no hay concurrencia al momento de escribir la salida principal.

Fragmento representativo del lanzamiento de hilos:

```
std::vector<PortResult> results(ports.size());
std::vector<std::thread> threads;
threads.reserve(ports.size());

for (size_t i = 0; i < ports.size(); ++i) {
    threads.emplace_back([baseTarget, port = ports[i], &results, i]() {
        scanSinglePort(baseTarget, port, results[i]);
    });
}

for (auto& t : threads) t.join();
```

La captura por lambda utiliza inicialización explícita (port = ports[i], baseTarget por copia) para garantizar que cada hilo recibe su propia copia del puerto y del destino base, en lugar de compartir una referencia que podría invalidarse o provocar lecturas incorrectas.

7.4 Manejo flexible de la especificación de puertos

Una de las mejoras explícitamente solicitadas para este parcial es el manejo más flexible de rangos. ArgumentParser acepta tres formatos intercambiables para la opción --ports:

- **Rango:** --ports 20-80, se expande al intervalo cerrado [20, 80].
- **Lista:** --ports 22,80,443, se interpreta como un conjunto explícito.
- **Puerto único:** --ports 80, se trata como un conjunto de un solo elemento.

La expansión produce en todos los casos un std::vector<int> ordenado y sin duplicados, que es lo que PortScanner consume. Se validan rangos fuera del intervalo válido TCP (0 a 65535), caracteres no numéricos, rangos invertidos (por ejemplo 80-20) y mezcla inválida de separadores. Ante cualquier error de parseo, el programa imprime un mensaje descriptivo y termina con código de salida distinto de cero.

7.5 Presentación de resultados

ResultFormatter imprime exclusivamente los puertos detectados como abiertos, en una tabla alineada mediante std::setw y acompañada del nombre probable del servicio. La identificación del servicio es estática: se apoya en un std::map<int, std::string> que asocia aproximadamente 17 puertos bien conocidos (SSH, HTTP, HTTPS, MySQL, PostgreSQL, FTP, SMTP, DNS, etc.) con sus nombres. Los puertos abiertos que no aparezcan en el mapa se etiquetan como "desconocido". Es importante señalar que esta aproximación no realiza lectura de banners, por

lo que constituye una heurística basada en el número de puerto y no una identificación real del servicio escuchando en él.

Al final del escaneo se imprime una línea de resumen con el total de puertos analizados, cuántos resultaron abiertos, cuántos cerrados y el tiempo total transcurrido medido con std::chrono::steady_clock.

Ejemplo de salida real para ./scanner --host 127.0.0.1 --ports 20-80:

```
Escaneando 127.0.0.1 (61 puertos)...

PUERTO   ESTADO    SERVICIO
22       OPEN      SSH
80       OPEN      HTTP

---------------------------------------
Total: 61 puertos | Abiertos: 2 | Cerrados: 59
Tiempo: 1.23s
```

Los puertos cerrados deliberadamente no se imprimen uno por uno para evitar saturar la consola en rangos grandes; únicamente su conteo aparece en la línea de resumen.

7.6 Pruebas y validación

Se realizaron las siguientes pruebas manuales para validar la correctitud y el comportamiento del escáner:

1. Localhost con distintos formatos de --ports. Se probaron los tres formatos de especificación: rango (20-80), lista (22,80,443) y puerto único (80). Los tres producen el formato de salida esperado y clasifican correctamente los puertos.

2. Localhost, 200 puertos concurrentes (./scanner --host 127.0.0.1 --ports 1-200). El escaneo completo tomó 0.005 s con un uso de CPU reportado de 226 %, lo que confirma paralelismo multi-core real: el sistema operativo distribuyó los 200 hilos entre varios núcleos en lugar de ejecutarlos secuencialmente en un solo núcleo. Esta es la evidencia principal de que el modelo de concurrencia está funcionando efectivamente a nivel de SO.

3. Host remoto (./scanner --host scanme.nmap.org --ports 20-100, 81 puertos). El escáner identificó correctamente 22 OPEN SSH y 80 OPEN HTTP. El tiempo total fue de 75 s, valor elevado que se explica por el timeout por defecto del sistema operativo para connect() sobre los puertos filtrados por el firewall del host remoto: el SO no termina la llamada hasta agotar su propio temporizador interno. Esta limitación se encuentra dentro del alcance esperado para este parcial, pues el timeout configurable es parte del entregable final.

4. Entradas inválidas. Se probaron múltiples casos degenerados: ejecución sin argumentos, --ports vacío, caracteres no numéricos, mezcla incoherente de - y ,, puertos mayores a 65535, y

un host no resoluble por DNS. Todos los casos emiten un mensaje de error claro y terminan con código de salida distinto de cero, sin llegar nunca a la fase de escaneo.

5. Comparación con nmap. Al ejecutar nmap -p 20-30 127.0.0.1 y ./scanner --host 127.0.0.1 --ports 20-30 sobre el mismo rango de 11 puertos, ambas herramientas clasifican los 11 puertos como cerrados — consistencia total en el rango probado. Los puertos que nmap reporta como filtered en hosts remotos se clasifican aquí como CLOSED, diferencia esperada dado que no se implementa detección diferenciada de RST vs timeout.

Limitaciones conocidas

Consistentes con el alcance del parcial y ya planeadas para el entregable final:

- No se distingue entre un puerto cerrado activamente (por un RST) y un puerto filtrado por un firewall; ambos se reportan como CLOSED.
- No existe timeout configurable; se utiliza el comportamiento por defecto del SO para connect(), lo que puede provocar tiempos de ejecución elevados en hosts con muchos puertos filtrados, como se observó en la prueba contra scanme.nmap.org.
- El esquema "un hilo por puerto" no es óptimo para rangos muy grandes (del orden de miles o decenas de miles de puertos); un thread pool con cola de trabajo sería más eficiente en ese régimen. Esta optimización queda propuesta para el entregable final.

8. Diagramas

8.1 Diagrama de clases

```
                      +--------------------+
                      |    ScannerApp      |
                      +--------------------+
                      | + run(argc, argv)  |
                      +---------+----------+
                                |
             +-----------------+-----------------+
             |                 |                 |
             v                 v                 v
 +---------------------+ +---------------+ +----------------------+
 |  ArgumentParser     | | PortScanner   | |  ResultFormatter     |
 +---------------------+ +---------------+ +----------------------+
 | + parse(argc, argv) | | + scan(host,  | | + print(host,        |
 |   : Args            | |   ports)      | |   results, elapsed)  |
 | - parsePortSpec()   | |   : vector<   | | - serviceFor(port)   |
 | - usage()           | |     PortResult| +----------------------+
 +---------------------+ |     >         |
                         +---------------+
```

```
Tipos compartidos (Types.hpp):
  struct Args        { host, ports }
  struct PortResult  { port, state, service }
  enum  PortState    { OPEN, CLOSED }
```

Responsabilidades por clase:

- **ScannerApp:** orquesta la ejecución del programa. No contiene lógica de negocio; únicamente coordina las llamadas a las demás clases y gestiona el ciclo de vida general.
- **ArgumentParser:** expone métodos estáticos puros. Convierte argv en una estructura Args validada. Ante error de parseo, termina el proceso con un mensaje claro al usuario.
- **PortScanner:** contiene la lógica de sockets y concurrencia. Es la única clase que interactúa con el sistema operativo a nivel de red (llamadas POSIX).
- **ResultFormatter:** expone métodos estáticos puros. Convierte un vector<PortResult> en salida formateada sobre std::cout.

8.2 Diagrama de flujo del programa

```
[Inicio]
    |
    v
+-------------------------------------+
| ScannerApp::run(argc, argv)         |
+------------------+------------------+
                   |
                   v
+-------------------------------------+
| ArgumentParser::parse()             |
|  - valida --host y --ports          |
|  - expande spec a vector<int>       |
+------------------+------------------+
                   |
                   v
+-------------------------------------+
| PortScanner::scan()                 |
|  1. getaddrinfo(host) -> sockaddr_in |
|  2. results.resize(ports.size())    |
|  3. for i in 0..N:                  |
|       spawn thread(scanSinglePort)  |
|  4. join() todos los hilos          |
+------------------+------------------+
                   |
                   v
```

```
  +-------------------------------------+
  | ResultFormatter::print()            |
  |  - tabla con puertos OPEN + servicio |
  |  - resumen (totales + tiempo)       |
  +------------------+------------------+
                     |
                     v
                   [Fin]
```

Detalle del cuerpo de cada hilo (scanSinglePort):

```
  [hilo creado]
       |
       v
  socket() --> ¿ok? --no--> [cerrMutex] cerr << "error" --> result = CLOSED --> fin
hilo
       |                                                                        ^
       v                                                                        |
  sin_port = htons(port)                                                        |
       |                                                                        |
       v                                                                        |
  connect() --> ¿ret == 0? --sí--> result = OPEN ------> close(sock) --------------+
                                                                                |
                                     --no--> result = CLOSED --> close(sock) --------+
```

9. Justificación técnica

C++17 y STL. Se eligió C++17 por su soporte nativo de std::thread, std::mutex, std::lock_guard, y por la posibilidad de usar capturas de lambda con inicialización explícita ([port = ports[i], ...]) que simplifican notablemente el paso de parámetros a los hilos. Alternativas como el uso directo de pthread introducirían más código repetitivo sin aportar valor pedagógico adicional en este alcance; al fin y al cabo, las abstracciones de la cabecera <thread> son envoltorios sobre pthread en macOS y Linux.

Sockets POSIX. El uso directo de socket(), connect(), getaddrinfo() y close() es deliberado: expone al estudiante al mismo contrato que el kernel ofrece a cualquier proceso, sin esconderlo detrás de una biblioteca de alto nivel. Esto es didácticamente importante en un curso de Sistemas Operativos, ya que permite razonar directamente sobre descriptores de archivo, estados del socket y códigos de error de la llamada al sistema.

Modelo de concurrencia "un hilo por puerto". Entre las tres alternativas consideradas — (a) escaneo secuencial, (b) un hilo por puerto, y (c) thread pool con cola de trabajo compartida — se eligió (b) por ser el mínimo funcional que introduce concurrencia real y permite demostrar explícitamente la sincronización con std::mutex. Un thread pool sería más eficiente para rangos muy grandes, pero requiere introducir std::condition_variable, una cola protegida por mutex, y

lógica de terminación (técnica poison pill o contador de tareas), lo cual excedería el alcance de este parcial. El modelo elegido escala razonablemente hasta aproximadamente 1000 hilos simultáneos en macOS, suficiente para los rangos de prueba contemplados.

Ausencia de carrera de datos en results. Aunque múltiples hilos escriben sobre el mismo std::vector<PortResult>, cada hilo escribe exclusivamente en results[i] con un i único por hilo. El estándar de C++ (§23.2.2 container.requirements.dataraces) permite escrituras concurrentes a elementos distintos de un contenedor siempre y cuando no se modifique la estructura del contenedor durante las operaciones (es decir, sin push_back, resize, etc.). Pre-dimensionar el vector con results(ports.size()) antes de lanzar los hilos garantiza esta precondición.

Mutex únicamente para std::cerr. La única sincronización explícita necesaria es proteger las escrituras a std::cerr desde múltiples hilos (por ejemplo, ante un fallo de socket()). std::cout no se utiliza dentro de los hilos, sino solo desde ResultFormatter::print() una vez que todos los hilos han sido unidos (join), por lo que no necesita protección alguna. Esta asimetría ilustra un principio general: sincronizar únicamente los recursos realmente compartidos, no "por si acaso".

Tabla estática de servicios. Se prefirió un std::map<int, std::string> con puertos bien conocidos sobre una lectura real de banners. Esto evita introducir latencia adicional por cada conexión abierta (que implicaría leer bytes del socket con su propio timeout asociado) y mantiene el foco del parcial en la concurrencia. La identificación real de servicios mediante lectura de banners y heurísticas basadas en expresiones regulares se contempla como mejora para el entregable final.

10. Entregas por parcial (planificación)

Primer parcial (25 de febrero de 2026) — entregado

- Un tercio del reporte escrito: introducción, resumen, objetivos, tecnologías, justificación tecnológica, problemática y público objetivo.
- Un tercio de la funcionalidad: programa que recibe una IP y un rango pequeño de puertos y realiza un escaneo básico sin concurrencia.
- Repositorio en GitHub creado con el proyecto inicial.

Segundo parcial (22 de abril de 2026) — entrega actual

- Otro tercio del reporte escrito: sección de desarrollo completa, diagramas de clases y de flujo, justificación técnica detallada de las decisiones de concurrencia.
- Otro tercio de funcionalidad: manejo flexible de rangos (rango, lista y puerto único), formato de salida mejorado con tabla de servicios y resumen, y concurrencia mediante std::thread con sincronización sobre std::cerr.
- Repositorio GitHub actualizado con la nueva arquitectura de cuatro clases.

Final de semestre (fecha por confirmar)

- Reporte completo en PDF con nombre en formato ID_Nombre_ApellidoPaterno.
- Proyecto funcionando con todas las funcionalidades definidas: timeouts configurables, distinción entre CLOSED y FILTERED, lectura opcional de banners, y posible migración a thread pool para rangos grandes.
- Repositorio GitHub completo y actualizado.